Modelling Infectious Disease Spread

One-page repo summary

What It Is

Flask app for exploring epidemic scenarios with an agent-based SIR simulator and ML models that estimate peak infections, outbreak duration, and total infections from user inputs.

Who It's For

Primary persona inferred from the repo: students, researchers, or demo users testing how parameter changes affect outbreak forecasts in a browser.

What It Does

- Collects seven outbreak parameters in a browser form.
- Returns ML-predicted peak infected population, outbreak duration, and total infected population.
- Exposes a JSON /predict API for programmatic predictions.
- Exposes a JSON /simulate API that returns SIR timelines and optional agent snapshots.
- Shows a results dashboard with KPI cards, charts, and a browser-side spatial animation.
- Validates inputs and exposes a simple /health endpoint.
- Includes scripts to generate training data and train/save regression models.

How It Works

Frontend: Jinja templates in /templates plus static CSS and Chart.js render the form and dashboard.
Backend: app.py serves /, /web-predict, /predict, /simulate, and /health via Flask + flask-cors.
ML layer: ml_utils.py builds a pandas feature row and loads three joblib models from outputs/models.
Simulation: simulator/simulation.py runs a seeded agent-based model and returns timeline + summary.
Data flow: user input -> validation -> model load/predict -> rendered results; web results also run the Python simulator for timeline data while the page animates a separate JavaScript ABM view.
Persistence/auth/deployment details: Not found in repo.

How To Run

1. Install dependencies: pip install -r requirements.txt
2. Start the app: python app.py
3. Open http://127.0.0.1:5000/
4. If model files are missing, run: python train_models.py --data-path outputs/simulation_training_data.csv

---

Evidence sources: README.md, app.py, ml_utils.py, simulator/simulation.py, train_models.py, templates/index.html, templates/results.html, requirements.txt.